

REPÚBLICA BOLIVARIANA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
CATEDRA: ING. DEL SOFTWARE.
ING. INFORMATICA



Planteamiento Metodológico para el Proyecto CampusConnect

Estudiante:

Daniel Reyna. C.I: 29.989.066

Docente:

Félix Marquez

Introducción.

Bueno, después de haber definido el Project Charter, el Lean Canvas y la WBS (que hicimos con un enfoque predictivo tipo Cascada), ahora toca decidir cómo vamos a ejecutar el proyecto en la práctica. La teoría nos dice que no existe una metodología única que sirva para todo; depende del contexto, la incertidumbre, los plazos y los stakeholders.

En el caso de CampusConnect, tenemos un alcance bastante claro (los módulos de mapa, calendario, directorio y notificaciones), un presupuesto fijo de 180k USD y un plazo inamovible de 16 semanas porque la universidad quiere la app para el próximo ciclo académico. Sin embargo, también sabemos que durante el desarrollo pueden surgir ajustes en los requisitos, por ejemplo, cambios en la disposición de los edificios o nuevas necesidades de los estudiantes. Por eso, en lugar de ir a un Cascada puro (que sería muy rígido) o a un Ágil puro (que podría descontrolar el alcance y el presupuesto), hemos optado por un enfoque híbrido Water-Scrum-Fall.

La idea es simple: usar lo mejor de ambos mundos. La fase inicial y final serán más estructuradas (tipo Cascada) para asegurar la planificación y el despliegue, mientras que el desarrollo central se hará con Scrum para ganar flexibilidad y retroalimentación temprana. En este documento les cuento cómo lo vamos a organizar.

Justificación de la Metodología Híbrida (Water-Scrum-Fall)

Como vimos en la unidad, el modelo híbrido es muy común en empresas grandes y en proyectos con restricciones contractuales. En nuestro caso:

- **La fase “Water”** nos permite tener un proyecto bien fundamentado desde el inicio: el Charter ya está firmado, los requisitos de alto nivel están claros y la WBS nos da una estructura de entregables. Esto es clave para que el patrocinador (el Vicerrectorado) tenga la seguridad de que no se van a disparar los costos.
- **La fase “Scrum”** nos da la capacidad de adaptarnos a cambios sin tener que rehacer toda la documentación. Por ejemplo, si durante el desarrollo los usuarios piden un filtro adicional en el directorio, podemos agregarlo al Product Backlog y priorizarlo en el siguiente Sprint sin romper el cronograma general.
- **La fase “Fall”** nos asegura que el producto final pase por todas las pruebas de integración, rendimiento y seguridad, y que la publicación en las tiendas se haga de forma controlada, tal como exige la universidad.

En resumen, este enfoque nos permite cumplir con la triple restricción: el alcance macro es fijo (lo que está en el Charter y la WBS), pero el detalle se puede ajustar; el tiempo y el costo son fijos (16 semanas y 180k), y la calidad se garantiza con las pruebas finales.

Fases del Ciclo de Vida Híbrido

Vamos a dividir las 16 semanas en tres grandes bloques, con sus respectivas actividades y artefactos.

3.1. Fase Water (Semanas 1-2) – Planificación y Diseño Inicial

Esta fase es totalmente predictiva. Aquí se completan todas las tareas que ya empezamos en los entregables anteriores y se deja todo listo para que el equipo de desarrollo pueda arrancar sin dudas.

Actividades principales:

- Finalizar el **Project Charter** y obtener todas las firmas (ya lo tenemos).
- Realizar el **análisis detallado de requisitos** basado en los 32 requisitos funcionales y los 7 no funcionales que identificamos. Esto incluye entrevistas adicionales con los directores de departamento para confirmar la disposición de los edificios y los horarios.
- Elaborar el **SRS** (Documento de Especificación de Requisitos) siguiendo la norma ISO/IEC/IEEE 29148, tal como se menciona en el material de la unidad.

- Diseñar la **arquitectura técnica** y la **base de datos** (ya tenemos los wireframes y el diseño UX/UI en la WBS, pero se refinan).
- Definir el **stack tecnológico** definitivo (por ejemplo, React Native para las apps, Node.js para el backend, PostgreSQL, etc.) y aprobar la infraestructura en la nube.
- Crear el **Plan de Pruebas Maestro** y la **Matriz de Trazabilidad de Requisitos**.
- Establecer el **Comité de Control de Cambios (CCB)** para gestionar cualquier desviación del alcance macro.

Entregables de esta fase:

- SRS aprobado
- Documento de Arquitectura (SAD)
- Plan de proyecto detallado (cronograma, presupuesto base)
- Plan de pruebas y de gestión de riesgos

Al final de la semana 2, hacemos un **Stage-Gate** con el patrocinador para validar que todo esté en orden y liberar los fondos para la fase de desarrollo.

3.2. Fase Scrum (Semanas 3-14) – Desarrollo Iterativo

Aquí es donde el equipo de desarrollo toma el control. Vamos a trabajar con **Sprints de 2 semanas** (ya que es un plazo corto y permite tener entregas rápidas). En total serán **6 Sprints** (semana 3 a la 14).

3.2.1. Roles del equipo

Como somos un equipo de 4 personas (más el líder que oficia de Product Owner parcial), adaptamos los roles de Scrum:

- **Product Owner (PO):** Daniel Reyna (el líder del proyecto). Él es el encargado de priorizar el Product Backlog, tomar decisiones sobre el alcance de cada Sprint y aprobar los incrementos. También es el enlace con el patrocinador.
- **Scrum Master:** Lo rotaremos entre los desarrolladores, pero inicialmente lo asumirá Arnaldo (quien hizo la WBS) para asegurar que se sigan las ceremonias y se eliminen impedimentos.
- **Developers:** Los 4 integrantes (incluido el PO) nos encargamos de la codificación, pruebas y diseño. Como todos tenemos habilidades variadas, podemos hacer pair programming cuando sea necesario, inspirados en XP.

3.2.2. El Product Backlog

A partir de los requisitos funcionales y la WBS, vamos a crear un **Product Backlog** con **Historias de Usuario** que cumplan el criterio INVEST. Por ejemplo:

- “Como estudiante, quiero ver un mapa interactivo del campus con los edificios etiquetados, para poder encontrar mi aula sin perderme.”
- “Como profesor, quiero recibir notificaciones push cuando se modifique el horario de mi examen, para estar al tanto.”

Cada historia tendrá una estimación en puntos de historia (usando la técnica de Planning Poker) y su prioridad la define el PO basándose en el valor de negocio y las dependencias técnicas.

El Backlog se refinará en las reuniones de **Backlog Grooming** que haremos cada semana.

3.2.3. Eventos de Scrum

- **Sprint Planning:** Al inicio de cada Sprint (2 horas), el equipo selecciona las historias que puede comprometerse a completar, definiendo el **Sprint Goal**.
- **Daily Scrum:** Todos los días, 15 minutos, para sincronizar avances y obstáculos.
- **Sprint Review:** Al final de cada Sprint (1 hora), mostramos el incremento funcional al PO y a algunos stakeholders (por ejemplo, un representante de estudiantes) para obtener feedback.
- **Sprint Retrospective:** Después de la Review (1 hora), analizamos cómo podemos mejorar como equipo.

3.2.4. Distribución de los Sprints

He propuesto la siguiente distribución tentativa, basada en los módulos de la WBS:

Sprint	Semanas	Objetivo principal	Entregable esperado
Sprint 1	3-4	Configuración del entorno, autenticación y gestión de usuarios (R1-R5)	Backend con registro/login funcional, base de datos desplegada.
Sprint 2	5-6	Mapa interactivo (R6-R10) y	API de mapas integrada, app

		primeros puntos de interés	móvil muestra el mapa con edificios.
Sprint 3	7-8	Calendario académico (R11-R16) y sincronización de eventos	Calendario con eventos oficiales y RSVP básico.
Sprint 4	9-10	Directorio universitario (R17-R21) y búsquedas	Directorio funcional con filtros, backoffice para gestionar datos.
Sprint 5	11-12	Notificaciones push y anuncios (R22-R25)	Sistema de notificaciones segmentadas, integración con Firebase.
Sprint 6	13-14	Integración completa, pruebas de regresión y refactorización	Aplicación integrada, tests automatizados, preparación para UAT.

Durante estos Sprints, también se irán desarrollando el Backoffice y las pruebas unitarias (como indica la WBS). La idea es que al final del Sprint 6 tengamos un producto completo y estable.

3.2.5. Prácticas técnicas adicionales (inspiradas en XP)

Para garantizar la calidad, vamos a adoptar algunas prácticas de Programación Extrema:

- **TDD (Test-Driven Development):** Escribir pruebas unitarias antes del código, especialmente en el backend y en la lógica de mapas.
- **Integración Continua:** Usar GitHub Actions para compilar y ejecutar pruebas automáticamente con cada commit.
- **Refactorización continua:** Mejorar el código sin cambiar su comportamiento, siempre que la suite de pruebas lo permita.

- **Programación en parejas** en tareas complejas (por ejemplo, la integración con el SDK de mapas).

Esto nos ayudará a mantener la deuda técnica baja y a cumplir con los tiempos de respuesta de 2 segundos que exige el RNF2.

3.3. Fase Fall (Semanas 15-16) – Pruebas de Sistema, Despliegue y Publicación

Aquí volvemos a un enfoque más predictivo y secuencial, porque ya no se pueden hacer cambios funcionales, solo correcciones.

Actividades:

- **Pruebas de integración y sistema:** Se ejecutan todas las pruebas no funcionales (rendimiento, carga, seguridad) que definimos en la WBS. Verificamos que el sistema soporte 500 usuarios concurrentes y que las consultas al mapa no superen los 2 segundos.
- **Pruebas de aceptación del usuario (UAT):** Invitamos a un grupo de estudiantes y profesores para que usen la app en un entorno de staging. Recogemos sus comentarios y corregimos errores críticos (Nivel 1 y 2).
- **Auditoría de seguridad y cumplimiento de datos:** Revisamos que se cumpla con la Ley de Datos (GDPR) y se hace un escaneo de vulnerabilidades.
- **Preparación de la publicación:** Generamos los binarios para iOS y Android, preparamos las fichas en App Store y Google Play, y subimos las capturas de pantalla, políticas de privacidad, etc.
- **Despliegue en producción:** Configuramos los servidores finales, balanceadores de carga, CDN y monitoreo. Hacemos el corte (cutover) y lanzamos oficialmente la app.
- **Capacitación al personal operativo** (soporte Nivel 1) y entrega de manuales de usuario.

Entregables finales:

- App publicada en ambas tiendas.
- Informe de pruebas UAT firmado por el patrocinador.
- Documentación de operaciones y lecciones aprendidas.
- Acta de cierre del proyecto.

Gestión de Cambios y Gobernanza en el Híbrido

Una de las claves del éxito de este modelo es que los cambios se manejan de manera diferente según la fase:

- Durante la **fase Water**, cualquier cambio en el alcance macro (por ejemplo, añadir un módulo de chat) debe pasar por el CCB y ser aprobado por el patrocinador, porque afectaría el presupuesto y el cronograma.
- Durante la **fase Scrum**, los cambios en el detalle (por ejemplo, agregar un filtro de búsqueda) se gestionan directamente en el Product Backlog; el PO decide si se incluyen en un Sprint futuro, siempre que no comprometan el objetivo de los Sprints ya planificados.
- En la **fase Fall** solo se aceptan correcciones de errores, no nuevas funcionalidades.

Además, usaremos la WBS como línea base de alto nivel, pero el equipo de desarrollo tendrá un **Sprint Backlog** detallado que se actualiza cada dos semanas. Esto evita el famoso “efecto sandía” (verde por fuera, rojo por dentro) porque realmente empoderamos al equipo para que tome decisiones técnicas durante los Sprints.

Ventajas y Posibles Riesgos de este Enfoque

Ventajas:

- Tenemos la seguridad de un plan inicial (Charter, WBS, presupuesto) que satisface a los directivos.
- El equipo puede adaptarse a cambios razonables sin paralizar el proyecto.
- La retroalimentación temprana de los usuarios (en las Sprint Reviews) nos permite corregir el rumbo antes de que sea muy tarde.
- Las pruebas finales aseguran que el producto cumpla con los estándares de calidad y seguridad exigidos.

Posibles riesgos y cómo mitigarlos:

- **Riesgo de que la fase Water se alargue:** Para evitarlo, hemos fijado un límite estricto de 2 semanas; si algo no está listo, se pospone para el Backlog.
- **Riesgo de que los Sprints no entreguen valor por sí solos:** Por eso hemos planeado que cada Sprint tenga un entregable funcional (por ejemplo, el mapa ya se puede ver al final del Sprint 2). Además, haremos demos con el PO cada dos semanas.

- **Riesgo de que el equipo no esté familiarizado con Scrum:** Haremos una pequeña capacitación interna la primera semana y el Scrum Master velará por que se sigan las reglas.

Conclusión

En mi opinión, este modelo híbrido es el más adecuado para CampusConnect porque combina lo mejor de ambos mundos: la planificación rigurosa que exige la universidad y la agilidad que necesitamos para construir un producto que realmente funcione y guste a los usuarios. Además, nos permite cumplir con las 16 semanas y el presupuesto de 180k sin tener que improvisar sobre la marcha.

Con este planteamiento, el equipo sabe qué hacer en cada fase, el patrocinador tiene visibilidad y control, y los estudiantes y profesores recibirán una aplicación de calidad que resuelve sus problemas de orientación y comunicación. Al final, lo que importa es entregar valor, y creo que este camino es el correcto.

Referencias

- Project Management Institute. (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)* – 7th ed.
- Schwaber, K. & Sutherland, J. (2020). *The Scrum Guide*.
- Beck, K. (2000). *Extreme Programming Explained*.
- West, D. (2011). *Water-Scrum-Fall Is the Reality of Agile for Most Enterprises*. Forrester.